

XML streams, over the connection created at the beginning of the interaction.

Before we get down to the code, we need to define some terms and underlying concepts that are involved with the use of the XMPP protocol. To write an XMPP client, we don't need a thorough understanding of the XML streams that are exchanged between client and server, since we use libraries that abstract away the complexities of the underlying protocol, and provide an API to us. In this article, we'll be using Python to write our code, and the `python-xmpp` library provides us a neat API. We need to understand the types of communication and interaction involved, however, so let's begin.

Note: The XML streams shown below as examples are **not** as they would actually be in a live XMPP session, since I have omitted attributes of some elements, and omitted portions of the XML stream that are not required to illustrate the concept. Since the `python-xmpp` library handles the nitty-gritty of building, sending, and receiving the XML streams, you don't need to memorise the XML samples—just look at them as illustrations, and **not** as code that you have to write yourself.

Resource

The XMPP client could be running on any of several types of computing devices, ranging from mobile phones and embedded devices to laptops and full-fledged desktops. The type of devices on which the client is running can be exposed in the *from* attribute in XML streams sent by the client, and this information is termed 'resource'. The *from* attribute is in the format *from*="userid@domain/resource". For example, a client running on an Android phone could send something like "userid@gmail.com/Android". This identifier can be very useful in many ways: administration tools that manage many clients could segregate the clients on this basis. The server could adjust its responses to the type of resource being used—for example, if connected from a mobile phone, the length of responses, the size of images, etc. could be held to a minimum to avoid long download times on a slow GPRS connection.

With this information exposed to the clients that buddies are running, it is then possible for clients to also treat different buddy resources in a different manner. For example, I might want to send the message "Hello Android guys!" to all my buddies who are connected from an Android device, but send "Hello, netbook guys!" to all buddies connected from netbooks.

Stanza

A stanza is an atomic command in XMPP, and one of the fundamental structures in the protocol layer. You can send an unlimited number of stanzas over an established connection between server and client.

Once an authenticated connection and XML stream is established between the server and the client, the client and the server can exchange (as part of the messaging service) three basic XML stanzas in their communications—`<message>`, `<presence>`, and `<iq>`. We'll look at each of those in turn.

Presence

As the word signifies, *presence* is a method by which a client/user notifies buddies and the XMPP server of its current status—whether the user is online or offline. When a client authenticates, it sends a presence notification stanza to the server, with some metadata that describes the client—the current status text, whether *Busy*, *Available* or *Away*, the resource parameter, user nickname, the client name, etc. When the server receives the presence stanza, it sends a copy of the data to all users who are in the buddy list of the user who sent the presence stanza. If you use the Pidgin messenger to connect to GTalk, hover the mouse pointer over an entry in your buddy list to see some of that metadata.

Here's a sample of an XML presence indication stanza:

```
<presence from="slynnu@slynnu.com/Android">
  <show>x</show>
  <status>Writing for LPIK</status>
</presence>
```

Message

Message is an XML stanza used to send messages between users. It looks like what's shown below:

```
<message from="slynnu@gmail.com/Home"
  to="slynnuguy@gmail.com"
  type="chat">
  <body>Hey, whats up ?</body>
  <subject>Query</subject>
</message>
```

IQ

IQ (Info/Query) is an XML stanza that is similar to GET and POST requests in the HTTP protocol. We use IQ to request for some information from the server, and collect the response for further use. If the request is invalid or cannot be processed, the server returns an error stanza. The XMPP protocol states that every *iq* stanza should contain an *id* attribute, whose value is generated by the client (the XMPP library used). This *id* attribute is returned in the *iq* response from the server, and can be used by the client to match the received response with a specific *iq* request it sent (useful in the case of the client sending multiple *iq* requests in a batch). Since the underlying details of generating a unique *id* are dealt with by the library (in our case, `python-xmpp`), we don't have to worry about that in our code.

For example, the following snippet is a roster query (more about that below) as sent to the server:

```
<iq from="slynnuguy@gmail.com"
  id="77"
  to="slynnuguy@gmail.com/Pidgin"
  type="get">
  <query xmlns="jabber:iq:roster">
</iq>
```